CHAPTER – X

MAP INTERFACE:

- Map is part of the Java Collection Framework, but NOT a subinterface of Collection.
- It stores data in key-value pairs.
- **KEY FEATURES:**
  - Keys are unique (no duplicates)
  - Values can be duplicated
  - One key maps to one value
  - Fast lookup using keys

SYNTAX:

Map<KeyType, ValueType>map=new HashMap<();

Common Implementations of Map

Implementation	Features
HashMap	Fast: no order maintained: allows one null key
LinkedHashMap	Maintains insertion order
TreeMap	Sorted by keys: no null key
Hashtable	Synchronized: no null key/value

EXAMPLE:

```
import java.util.HashMap;
public class HashMapExample {
   public static void main(String[] args) {
      HashMap<Integer, String> students = new HashMap<>();
      students.put(101, "Shalini");
      students.put(102, "Rahul");
```

```
        students.put(103, "Priya");
        System.out.println(students);
        System.out.println("Student with ID 102: " + students.get(102));
        students.remove(103);
        System.out.println("After removing ID 103: " + students);
    }
}
```

QUEUE INTERFACE:

- Queue is a subinterface of Collection in Java.
- It represents a collection designed for holding elements before processing, typically in FIFO (First In, First Out) order.
- **KEY FEATURES OF QUEUE:**
  - FIFO order (first element added is the first removed
  - Duplicates allowed

Common Implementations of Queue

Implementation	Features
LinkedList	Implements Queue; allows null
PriorityQueue	Sorted order; no null allowed
ArrayDeque	Fast; no null allowed; recommended replacement for Stack

✅ Syntax

```java
Queue<Type> queue = new LinkedList<>();
```

or

```java
Queue<Type> queue = new PriorityQueue<>();
```

EXAMPLE:

```
import java.util.*;
public class QueueDemo {
   public static void main(String[] args) {
      Queue<Integer> q = new LinkedList<>();
      q.add(10);
      q.add(20);
      q.add(30);
      System.out.println(q);
      q.remove();
      System.out.println(q);
   }
}
```

EXAMPLE – 2 PRIORITY QUEUE

```
import java.util.PriorityQueue;
public class PriorityQueueDemo {
   public static void main(String[] args) {
      PriorityQueue<Integer> q = new PriorityQueue<>();
      q.add(30);
      q.add(10);
      q.add(20);
      System.out.println("Queue: " + q);
      System.out.println("Removed: " + q.poll());
      System.out.println("Queue after poll: " + q);
   }
```

}

Method	Description
`add()`	Adds element (throws exception if fails)
`offer()`	Adds element (returns false if fails)
`poll()`	Removes head; returns null if empty
`remove()`	Removes head; throws exception if empty
`peek()`	Returns head without removing
`element()`	Returns head; throws exception if empty

CURSOR IN JAVA:

- Java provides three types of cursors for iterating (looping) through collections:
  - Iterator
  - ListIterator
  - Enumeration
- These are used to traverse elements of collections like List, Set, Vector, etc.

ITERATOR:

- Iterator is a universal cursor used to traverse elements of any collection (List, Set, Queue).
- Works with all collection classes
- Moves forward only (one direction)
- Allows remove() during iteration
- Modern replacement for Enumeration
- **METHODS:**
  - hasNext()
  - next()
  - remove()

EXAMPLE:

```
import java.util.Iterator;
import java.util.PriorityQueue;
public class IteratorDemo {
   public static void main(String[] args) {
      PriorityQueue<Integer> q = new PriorityQueue<>();
      q.add(10);
      q.add(20);
      q.add(30);
      Iterator<Integer> it = q.iterator();
      while (it.hasNext()) {
         System.out.println(it.next());
      }
   }
}
```

LISTITERATOR:

- ListIterator is a cursor used only for List (ArrayList, LinkedList) and supports two-way traversal.
- Works only with List
- Moves forward and backward
- Can add(), remove(), set() elements
- More powerful than Iterator
- **METHODS:**
  - hasNext(), next()
  - hasPrevious(), previous()
  - add(), remove(), set()

EXAMPLE:

```
import java.util.*;
public class ListIteratorExample {
   public static void main(String[] args) {
      ArrayList<String> list = new ArrayList<>();
      list.add("Apple");
      list.add("Banana");
      list.add("Mango");
      ListIterator<String> it = list.listIterator();
      while (it.hasNext()) {
         System.out.println(it.next());
      }
   }
}
```

ENUMERATION:

- Enumeration is a legacy cursor used only for old classes like:
  - Vector
  - Stack
  - Hashtable
- Works only with legacy classes
- Moves forward only
- Can't remove elements
- Oldest cursor (before Iterator)
- **METHODS:**
  - hasMoreElements()
  - nextElement()

EXAMPLE:

```
import java.util.*;
public class EnumerationExample {
   public static void main(String[] args) {
      Vector<String> v = new Vector<>();
      v.add("Apple");
      v.add("Banana");
      v.add("Mango");
      Enumeration<String> e = v.elements();
      while (e.hasMoreElements()) {
         System.out.println(e.nextElement());
      }
   }
}
```